Ejercicios · UD02 · Sesión del 14/10/2026

Desarrollo de Interfaces · 2.º DAM

Módulo	Desarrollo de Interfaces · 2.º DAM
Unidad	UD02 · Generación de interfaces naturales de usuario
Sesión	Miércoles 14/10/2026 · 18:15–21:00
Contenido trabajado	Diapositivas 1–18: apertura y Bloque 1 · Aprendizaje automático y entrenamiento (apartado 1 del PDF)
Temas	Cuándo se justifica el aprendizaje automático · ciclo de vida de un modelo · integrar modelos ya entrenados · separar la interfaz de la lógica de reconocimiento (CommandInterpreter) · entrenamiento, validación y sobreajuste · respuestas seguras ante predicciones dudosas · partición 60/20/20
Puntuación	10 puntos: Ejercicio 1 (3) · Ejercicio 2 (4) · Ejercicio 3 (3)

Ejercicio 1 · ¿Modelo o regla? Decidir con criterio (3 puntos)

Trabajas en una empresa que desarrolla aplicaciones para distintos clientes. Te llegan estas cuatro peticiones:

- Caso A. Comprobar que el DNI escrito en un formulario tiene 8 números y una letra.
- Caso B. Reconocer la orden «¡para!» dicha en voz alta por operarios en un taller con maquinaria ruidosa.
- Caso C. Detectar si el usuario escribe la palabra «salir» en un campo de comandos de texto.
- Caso D. Reconocer qué ejercicio de mano está haciendo un paciente en una app de rehabilitación con la cámara del móvil.

a) Para cada caso, indica si está justificado usar aprendizaje automático o basta con una regla programada. Justifica tu respuesta con el criterio visto en clase: ¿una regla manual sería insuficiente, frágil o imposible de mantener? *(1,2 puntos)*

- Caso A: Regla programada. Un DNI sigue un patrón rígido y exacto comprobable con expresiones regulares; un modelo sería innecesario.

- Caso B: Aprendizaje automático. El audio en entornos reales varía infinitamente por el ruido, acentos y volumen, lo que hace imposible crear reglas manuales exhaustivas.

- Caso C: Regla programada. La comparación de texto exacto o normalizado ("salir") es determinista y no requiere de inferencia.

- Caso D: Aprendizaje automático. La interpretación visual de píxeles capturados por una cámara para deducir posturas varía según la luz, el ángulo y el paciente, requiriendo generalización.

b) Una app de banca integra un reconocedor de voz ya entrenado. Recibe dos órdenes, ambas con una confianza de 0,62: «transfiere 500 euros a Laura» y «abre la ayuda». Explica qué debería hacer la interfaz en cada caso y por qué no deben tratarse igual. *(1 punto)*

- "Transfiere 500 euros a Laura": La interfaz debe bloquear la acción o pedir confirmación manual explícita. El nivel de confianza es demasiado bajo para una operación financiera crítica que requiere certeza casi absoluta.

- "Abre la ayuda": La interfaz puede ejecutar la acción o mostrar la ayuda discretamente, ya que es una operación segura y sin impacto destructivo.

c) Para el caso B, enumera cuatro condiciones reales con las que habría que probar el modelo antes de desplegarlo y explica por qué no basta con que funcione en el portátil del desarrollador. *(0,8 puntos)*

1º Ruido ambiental de maquinaria en funcionamiento.

2º Diferentes acentos, timbres de voz y velocidad de habla de los operarios.

3º Uso de equipos de protección (mascarillas, protectores) que atenúen el audio.

4º Distancia variable entre el operario y el micrófono. El portátil del desarrollador ofrece un entorno estéril y silencioso que no representa la distribución de datos reales a los que se enfrentará el modelo en producción.

Ejercicio 2 · Un intérprete de comandos más robusto (4 puntos)

Partimos del clasificador visto en clase (diapositiva 8):

```
public class IntentClassifier {
    public String classify(String text) {
        String normalized = text.toLowerCase().trim();

        if (normalized.contains("abrir") && normalized.contains("camara")) {
            return "OPEN_CAMERA";
        }
        if (normalized.contains("activar") && normalized.contains("ayuda")) {
            return "ENABLE_HELP_MODE";
        }
        return "UNKNOWN_INTENT";
    }
}
```

a) Sin ejecutarlo, indica qué devuelve classify para cada una de estas entradas y explica el motivo: «Abrir Camara», « ABRIR la cámara », «activa la ayuda por favor», «necesito ayuda». ¿Qué ocurre si recibe null? *(1 punto)*

«Abrir Camara» devuelve OPEN_CAMERA (coinciden "abrir" y "camara" en minúsculas).

« ABRIR la cámara » devuelve UNKNOWN_INTENT (la palabra "cámara" contiene tilde, y el código estricto busca "camara").

«activa la ayuda por favor» devuelve UNKNOWN_INTENT (contiene "activa", pero el código exige la palabra "activar").

«necesito ayuda» devuelve UNKNOWN_INTENT (falta la condición de contener "activar").

Si recibe null, lanzará una excepción NullPointerException al intentar ejecutar .toLowerCase().

Ahora vas a aplicar el contrato de la diapositiva 10:

```
public interface CommandInterpreter {
    String detectIntent(String inputText);
}
```

b) Implementa en Java la clase RobustCommandInterpreter, que implemente CommandInterpreter y cumpla estas reglas: *(1,5 puntos)*

- Si la entrada es null o está en blanco, devuelve EMPTY_INPUT.
- Normaliza el texto: minúsculas, sin espacios al principio ni al final y sin tildes. Pista: java.text.Normalizer con la forma NFD y después elimina las marcas diacríticas con replaceAll("\\p{M}", "").
- OPEN_CAMERA si el texto contiene «camara» y además «abrir», «abre» o «enciende».
- ENABLE_HELP_MODE si contiene «ayuda».
- CLOSE_APP si contiene «salir».
- UNKNOWN en cualquier otro caso.

c) Escribe un método main que pruebe tu clase con al menos seis entradas (incluidas null, una cadena en blanco y una frase con tildes) y presenta los resultados en una tabla con las columnas: entrada, intención esperada, intención obtenida. *(0,8 puntos)*

```
$ javac CommandInterpreter.java RobustCommandInterpreter.java TestCommandInterpreter.java
$ java TestCommandInterpreter

Entrada                         | Intencion esperada | Intencion obtenida
--------------------------------------------------------------------------
null                            | EMPTY_INPUT        | EMPTY_INPUT
"   "                           | EMPTY_INPUT        | EMPTY_INPUT
"Abrir Camara"                  | OPEN_CAMERA        | OPEN_CAMERA
"  ABRIR la cámara  "           | OPEN_CAMERA        | OPEN_CAMERA
"Enciende la CÁMARA"            | OPEN_CAMERA        | OPEN_CAMERA
"abre la camara"                | OPEN_CAMERA        | OPEN_CAMERA
"activa la ayuda por favor"     | ENABLE_HELP_MODE   | ENABLE_HELP_MODE
"necesito ayuda"                | ENABLE_HELP_MODE   | ENABLE_HELP_MODE
"quiero salir"                  | CLOSE_APP          | CLOSE_APP
"¿qué hora es?"                 | UNKNOWN            | UNKNOWN

Pruebas correctas: 10 de 10
```

d) Busca al menos una frase razonable que tu intérprete clasifique mal y explica por qué falla. Después explica, en 3–5 líneas, por qué la pantalla de la aplicación no tendría que cambiar si mañana sustituyes tu clase por un modelo de PLN. *(0,7 puntos)*

Frase que falla: *"no quiero abrir la camara nunca". El sistema devolverá OPEN_CAMERA porque contiene las palabras clave, pero no es capaz de procesar la negación contextual.*

Aislamiento de la interfaz: *La pantalla no debe cambiar porque el uso de la interfaz CommandInterpreter garantiza el desacoplamiento. El resto del sistema solo consume el contrato definido (detectIntent), ignorando si la lógica subyacente se procesa mediante sentencias condicionales o un modelo complejo de PLN.*

Ejercicio 3 · Entrenar, validar y no engañarse (3 puntos)

Partimos del código de partición de datos visto en clase (diapositivas 16–17):

```
List<String> shuffled = new ArrayList<>(samples);
Collections.shuffle(shuffled, new Random(42));

int trainSize = (int) Math.round(shuffled.size() * 0.6);
int validationSize = (int) Math.round(shuffled.size() * 0.2);

List<String> trainSet = new ArrayList<>(shuffled.subList(0, trainSize));
List<String> validationSet = new ArrayList<>(
        shuffled.subList(trainSize, trainSize + validationSize));
List<String> testSet = new ArrayList<>(
        shuffled.subList(trainSize + validationSize, shuffled.size()));
```

a) Completa la tabla aplicando exactamente ese cálculo e indica si algún conjunto queda vacío. *(1,2 puntos)*

N.º de ejemplos	Entrenamiento	Validación	Prueba	¿Algún conjunto vacío?
2	1	0	1	Si (Validación)
6	4	1	1	No
9	5	2	2	No
12	7	2	3	No
25	15	5	5	No

b) Las frases de ejemplo vienen ordenadas por intención: primero todas las de «abrir», después las de «cerrar» y al final las de «ayuda». ¿Qué pasaría si eliminas la línea Collections.shuffle? ¿Para qué sirve el 42? *(0,6 puntos)*

Si se elimina `Collections.shuffle`, el conjunto de entrenamiento (el 60% inicial) acapararía exclusivamente las primeras intenciones del listado ordenado ("abrir" y "cerrar"). El modelo jamás aprendería a reconocer la intención "ayuda", garantizando su fracaso. El número `42` actúa como una semilla constante que asegura que la aleatoriedad sea reproducible en ejecuciones futuras.

c) ¿Por qué cada subList se envuelve en new ArrayList<>(...)? ¿Qué «fallo silencioso» evita añadir una comprobación de que ningún conjunto está vacío? *(0,6 puntos)*

Se envuelven en `new ArrayList<>(...)` porque `subList` solo devuelve una vista superficial enlazada a la lista original, lo que podría generar errores de concurrencia o modificaciones indeseadas cruzadas. Al evitar conjuntos vacíos, se previene el "fallo silencioso" matemático, como obtener divisiones por cero al calcular métricas (porcentajes de acierto).

d) Un compañero te dice: «Mi modelo de gestos acierta el 99 % con los datos de entrenamiento y el 61 % con los de validación, así que ya está listo». ¿Qué le responderías? Usa el término técnico adecuado. *(0,6 puntos)*

*Le respondería que su modelo sufre de **sobreajuste (overfitting)**. Ha memorizado perfectamente los datos de entrenamiento (99%), pero es incapaz de generalizar los patrones subyacentes, lo que se refleja en su pobre rendimiento frente a los datos nuevos del conjunto de validación (61%).*

Entrega

- Un documento (PDF) con las respuestas de los ejercicios 1 y 3 y de los apartados 2a, 2c y 2d. Las respuestas deben ser razonadas: no basta con poner el resultado, explica por qué.
- Los archivos .java del ejercicio 2 (RobustCommandInterpreter.java, CommandInterpreter.java y la clase con el main), o el proyecto comprimido.
- Una captura de la salida por consola de tu main.
- Plazo: antes de la próxima sesión (viernes 16/10/2026), salvo que el profesor indique otra fecha.